

Engenharia de Software Moderna

Cap. 5 - Princípios de Projeto

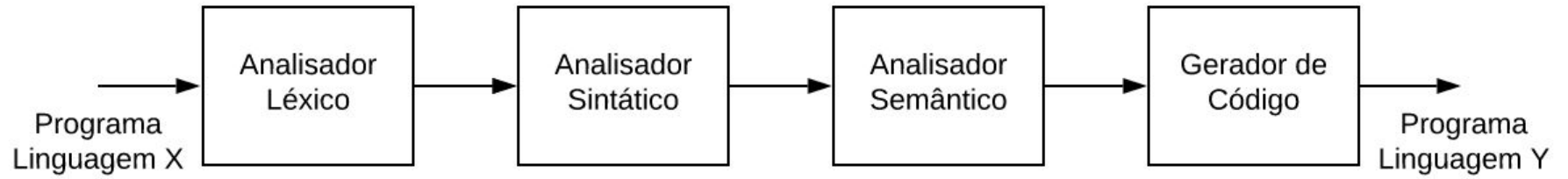
Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

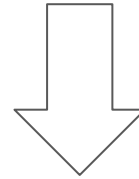
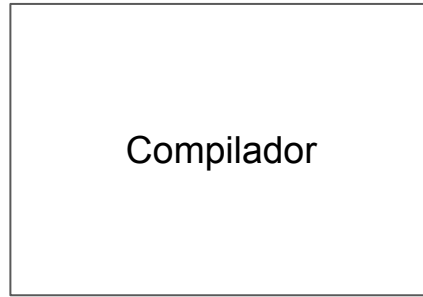
Licença CC-BY; permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Como implementar um compilador?

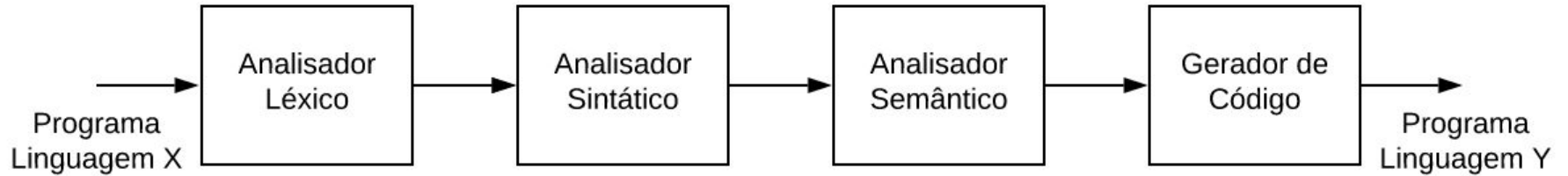




O que é design de software?



design

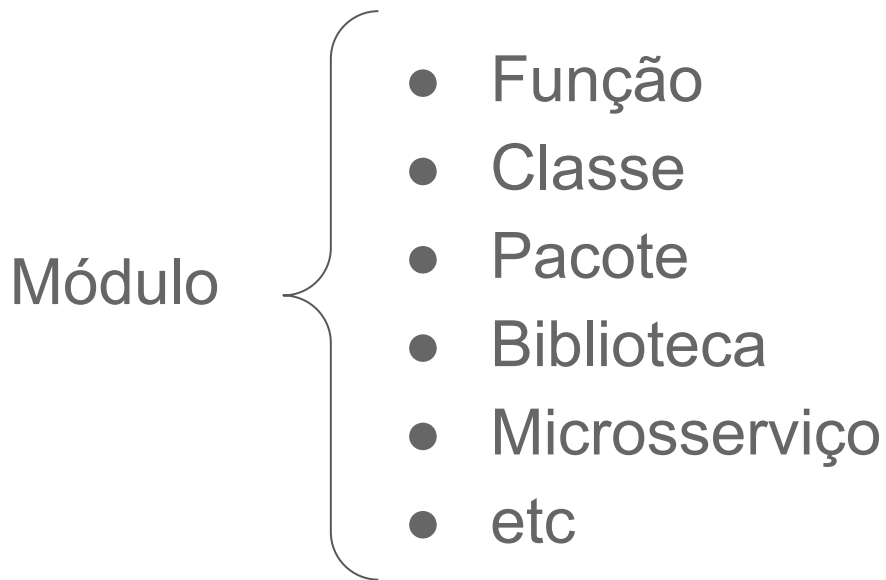


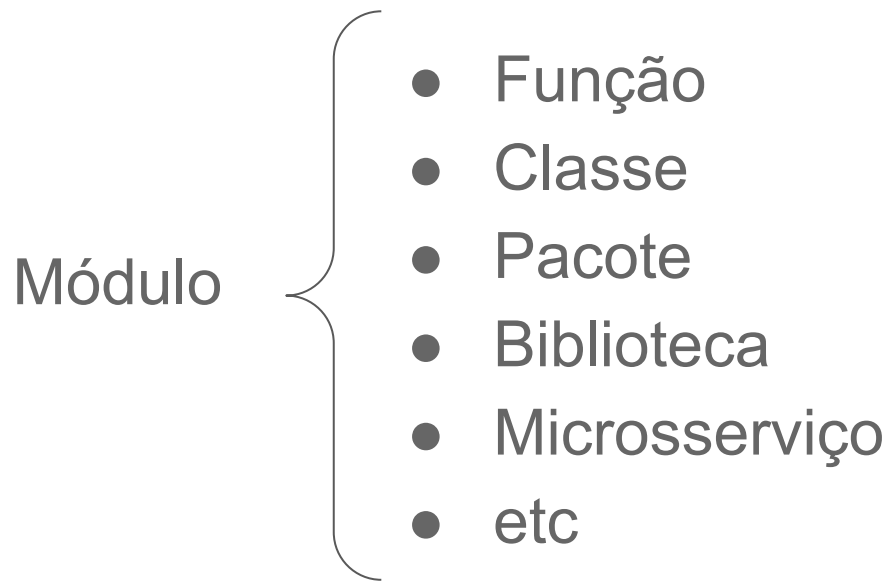
Projeto (Design) de Software

- Dividir para conquistar
- Quebrar um "problema grande" em partes menores

Módulos

- Nome que se dá em design para tais "partes menores"





Módulo = interface + implementação

O que vamos estudar?

Propriedades de "bons projetos" de software

- Integridade Conceitual
- Ocultamento de Informação
- Coesão
- Acoplamento

Princípios de Projeto

- Responsabilidade Única
- Segregação de Interfaces
- Prefira Interfaces a Classes
- Aberto/Fechado
- Demeter
- Substituição de Liskov

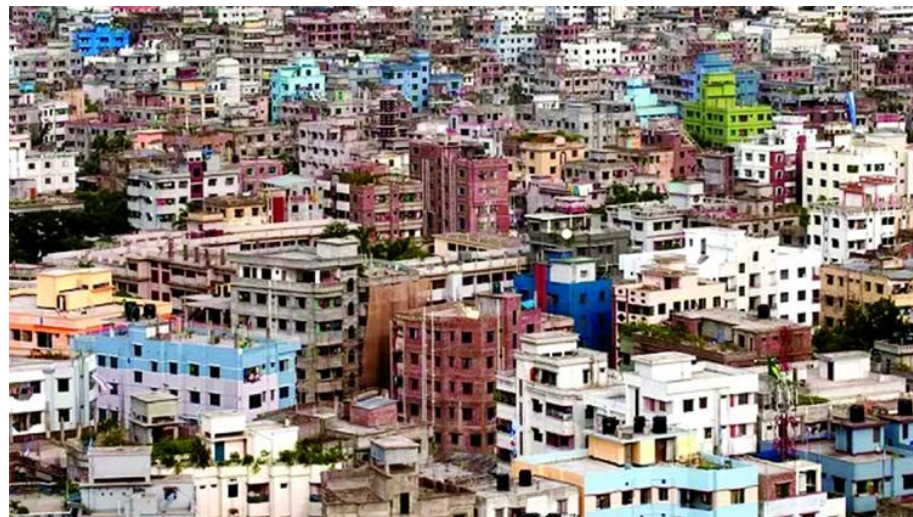
Propriedades de Projeto

Integridade Conceitual

Integridade Conceitual em 1 slide



Exemplo



Contra-Exemplo

Por que falta integridade conceitual nesses slides?

Licença CC-BY: permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Engenharia de Software Moderna

Cap. 4 - Modelos

Prof. Marco Tulio de Oliveira Valente

<https://engsoftmoderna.info>, @engsoftmoderna

1

Engenharia de Software Moderna

Cap. 5 - Princípios de Projeto

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>, @engsoftmoderna

Licença CC-BY: permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

2

Integridade Conceitual vale para:

- Funcionalidades
- Interface com usuário
- Projeto
- Implementação
- etc

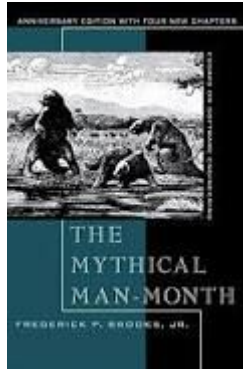
Exemplo (referente a interface com o usuário)

- Botão "sair" é idêntico em todas as telas
- Se um sistema usa tabelas para apresentar resultados, todas as tabelas têm o mesmo leiaute
- Todos os resultados são mostrados com 2 casas decimais

Exemplos (em nível de projeto/código)

- Todas as variáveis seguem o mesmo padrão de nomes
 - **contra-exemplo:** `nota_total` vs `notaMedia`
- Todas as páginas usam o mesmo framework (mesma versão)
- Se um problema é resolvido com uma estrutura de dados X, todos os problemas parecidos também usam X

Integridade conceitual é a consideração mais importante no projeto de sistemas -- Fred Brooks



Motivo: integridade conceitual facilita
implementação e manutenção de um sistema

Ocultamento de Informação

(Information Hiding)

Origem do conceito (David Parnas, 1972)

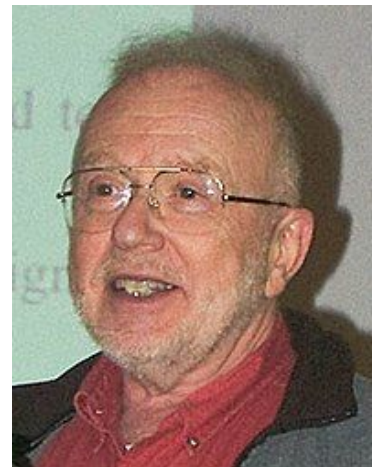
On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time. The effectiveness of a “modularization” is dependent upon the criteria used in dividing the system into modules. A system design problem is presented and

Introduction

A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:¹



```
import java.util.Hashtable;

public class Estacionamento {

    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```





```
import java.util.Hashtable;

public class Estacionamento {

    public Hashtable<String, String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.veiculos.put("TCP-7030", "Uno");
        e.veiculos.put("BNF-4501", "Gol");
        e.veiculos.put("JKL-3481", "Corsa");
    }
}
```

Problema: clientes precisam manipular uma estrutura de dados interna da classe, para estacionar um veículo

Comparação com um sistema manual de estacionamento



Problema

- Classes precisam de um pouco de "privacidade"
- Até para evoluir de forma independente dos clientes
- Código anterior: clientes manipulam a hashtable

Agora uma versão com ocultamento de
informação

```
import java.util.Hashtable;
```



```
public class Estacionamento {
```

1

```
    private Hashtable<String,String> veiculos;
```

```
    public Estacionamento() {
```

```
        veiculos = new Hashtable<String, String>();
```

```
    }
```

```
    public void estaciona(String placa, String veiculo) {
```

```
        veiculos.put(placa, veiculo);
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Estacionamento e = new Estacionamento();
```

```
        e.estaciona("TCP-7030", "Uno");
```

```
        e.estaciona("BNF-4501", "Gol");
```

```
        e.estaciona("JKL-3481", "Corsa");
```

```
    }
```

```
}
```

```
import java.util.Hashtable;
```

```
public class Estacionamento {
```



```
    private Hashtable<String,String> veiculos;
```

```
    public Estacionamento() {
```

```
        veiculos = new Hashtable<String, String>();
```

```
    }
```

```
    public void estaciona(String placa, String veiculo) {
```

```
        veiculos.put(placa, veiculo);
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Estacionamento e = new Estacionamento();
```

```
        e.estaciona("TCP-7030", "Uno");
```

```
        e.estaciona("BNF-4501", "Gol");
```

```
        e.estaciona("JKL-3481", "Corsa");
```

```
    }
```

```
}
```

```
import java.util.Hashtable;

public class Estacionamento {

    private Hashtable<String,String> veiculos;

    public Estacionamento() {
        veiculos = new Hashtable<String, String>();
    }

    public void estaciona(String placa, String veiculo) {
        veiculos.put(placa, veiculo);
    }

    public static void main(String[] args) {
        Estacionamento e = new Estacionamento();
        e.estaciona("TCP-7030", "Uno");
        e.estaciona("BNF-4501", "Gol");
        e.estaciona("JKL-3481", "Corsa");
    }
}
```



Resultado: classe Estacionamento fica livre para alterar a sua estrutura de dados interna

Ocultamento de Informação

- Classes devem ocultar detalhes internos de sua implementação (usando modificador **private**)
- Principalmente aqueles sujeitos a mudanças
- Adicionalmente, interface da classe deve ser estável
- **Interface** = conjunto de métodos públicos de uma classe

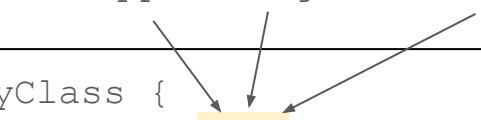
Significados da palavra interface

- Interface: conjunto de métodos públicos de uma classe
- Interface: construção de uma linguagem (palavra reservada)
- Existe ainda um terceiro significado:
 - Interface com o usuário (UI), interface com o usuário gráfica (GUI), interface mobile, etc
 - Fora do escopo deste curso

Interface em Java

```
interface Chat {  
    void sendMsg(String);  
}  
  
class WhatsApp implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}  
  
class Telegram implements Chat {  
    public void sendMsg(String m) {  
        ...  
    }  
}
```

WhatsApp Telegram etc



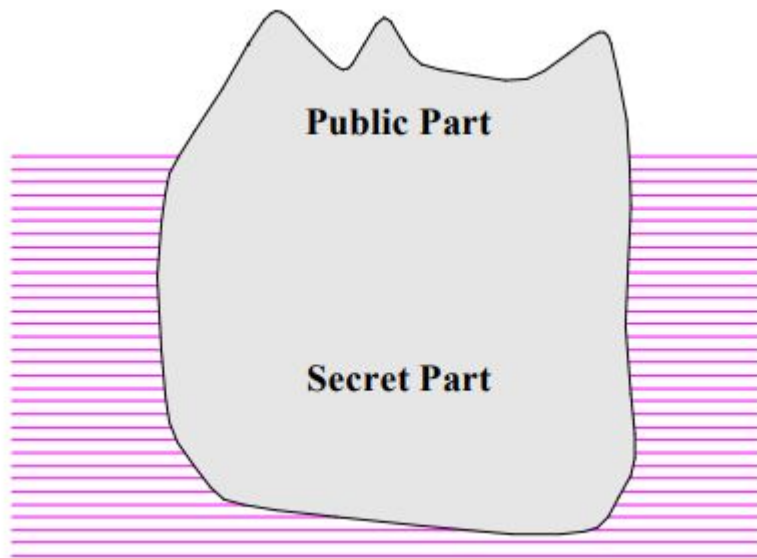
```
class MyClass {  
    public void f(Chat chat) {  
        ...  
        chat.sendMsg(...);  
        ...  
    }  
}
```

Código mantido por um time Y

Mesmo que uma classe não implemente uma interface (palavra reservada), ela possui uma interface (seus métodos públicos)

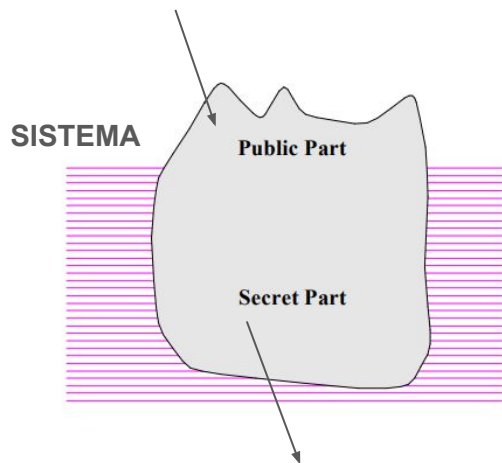
Bom módulos são semelhantes a icebergs

(pequena parte pública e visível; grande parte submersa e privativa)



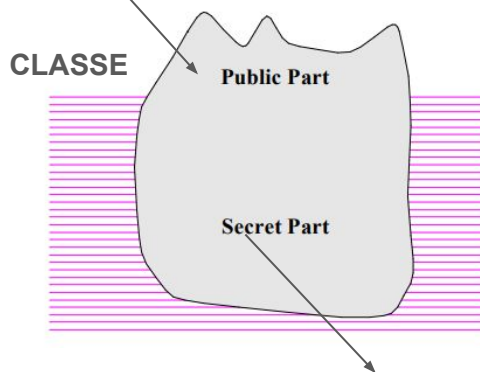
Generalizando a metáfora para sistemas, classes e funções

API (REST, gRPC, GraphQL)



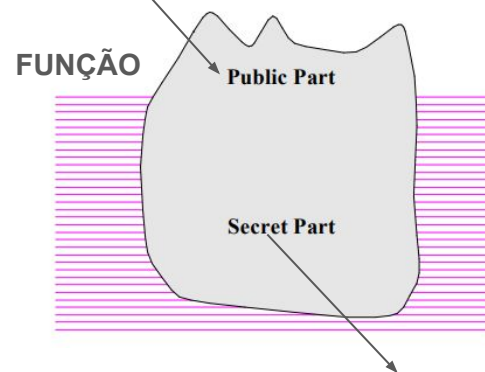
Sistema completo

Assinatura dos métodos públicos



Implementação de métodos públicos; métodos privados

Assinatura de uma função ou método



Implementação de uma função ou método

Outro nome: encapsulamento

- Alguns autores, preferem o nome encapsulamento, mas com o mesmo significado de ocultamento de informação

Encapsulation

See information hiding.

Glossário do livro Object-Oriented Software Construction.
Bertrand Meyer (p. 1195)

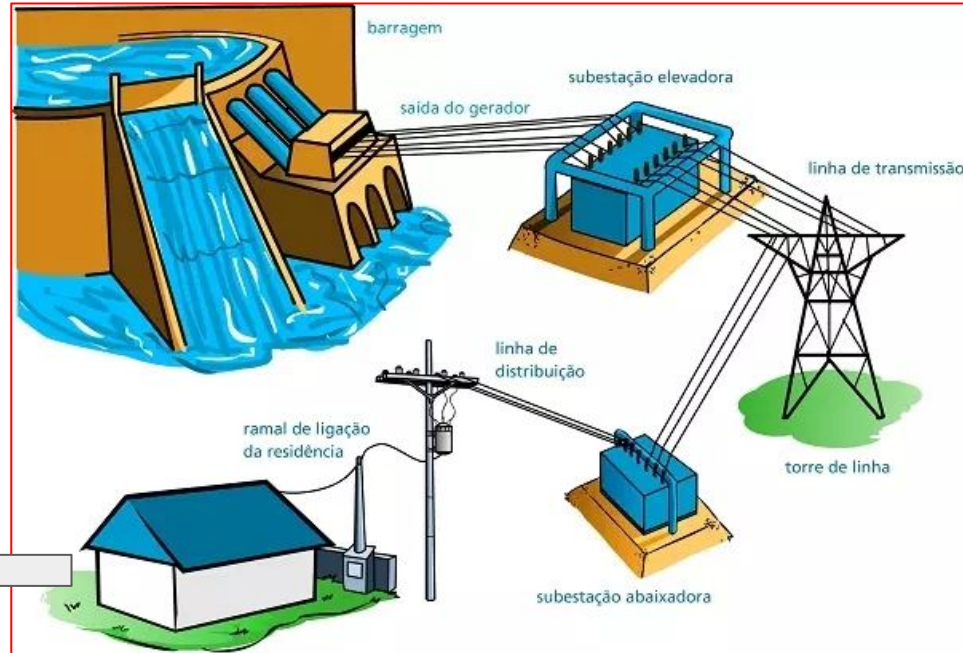
Information Hiding em 1 slide

Implementação

Interface



110 volts



Coesão

Coesão

- Uma classe deve ter uma única função, isto é, oferecer um único serviço
- Vale também para outras unidades: funções, métodos, pacotes, etc.

Contra-exemplo 1

```
float sin_or_cos(double x, int op) {  
    if (op == 1)  
        "calcula e retorna seno de x"  
    else  
        "calcula e retorna cosseno de x"  
}
```



Contra-exemplo 2

```
class Estacionamento {  
    ...  
    private String nome_gerente;  
    private String fone_gerente;  
    private String cpf_gerente;  
    private String endereco_gerente;  
    ...  
}
```



Exemplo

```
class Stack<T> {  
    boolean empty() { ... }  
    T pop() { ... }  
    push (T) { ... }  
    int size() { ... }  
}
```



Todos esses métodos manipulam os elementos da Pilha

Acoplamento

Acoplamento

- Nenhuma classe é uma ilha ...
- Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)
- A questão principal é a **qualidade** desse acoplamento
- Dois tipos:
 - Acoplamento aceitável ("bom")
 - Acoplamento ruim

Acoplamento Aceitável

- Classe A usa uma classe B e:
 - B oferece um serviço útil para A
 - B possui uma interface estável
 - A somente chama métodos da interface de B

```
import java.util.Hashtable;
```

```
public class Estacionamento {
```

```
    private Hashtable<String,String> veiculos;
```

```
    public Estacionamento() {  
        veiculos = new Hashtable<String, String>();  
    }
```

```
    public void estaciona(String veiculo, String placa) {  
        veiculos.put(veiculo, placa);  
    }
```

```
    public static void main(String[] args) {  
        Estacionamento e = new Estacionamento();  
        e.estaciona("TCP-7030", "Uno");  
        e.estaciona("BNF-4501", "Gol");  
        e.estaciona("JKL-3481", "Corsa");  
    }
```

```
}
```

Classe Estacionamento depende (está acoplada) à classe Hashtable, mas esse acoplamento é aceitável

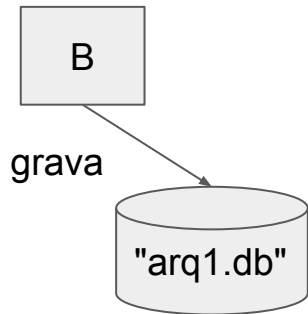


Acoplamento Ruim

- Classe A usa uma classe B:
 - a. Mas interface da classe B é **instável**
 - b. Ou então o uso **não** ocorre via interface de B

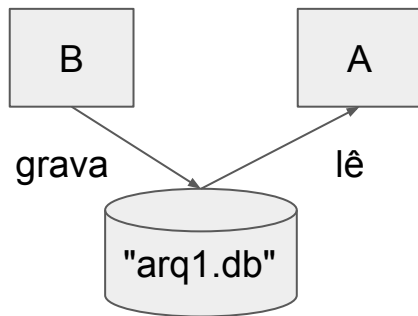
Como uma classe A estar acoplada a uma classe B
sem ser via a interface de B?

```
class B {  
    private void g() {  
        int total;  
        // computa valor de total  
        File f = File.open("arq1.db");  
        f.writeInt(total);  
        ...  
        f.close();  
    }  
}
```



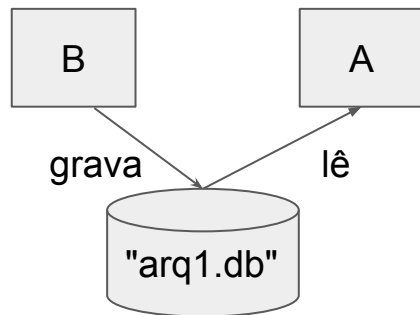
```
class B {  
    private void g() {  
        int total;  
        // computa valor de total  
        File f = File.open("arq1.db");  
        f.writeInt(total);  
        ...  
        f.close();  
    }  
}
```

```
class A {  
  
    private void f() {  
        int total;  
        ...  
        File f = File.open("arq1.db");  
        total = f.readInt();  
        ...  
    }  
}
```



Problema: Acoplamento ruim

- Mudanças internas em B podem impactar A
- Exemplo: B pode mudar o formato do arquivo ou mesmo deixar de salvar o dado que é lido por A



Na literatura, esse tipo de acoplamento é chamado também de acoplamento evolutivo ou lógico

Como resolver esse problema?
Como tornar um acoplamento ruim em bom?

Tornando acoplamento ruim em bom

```
class B {  
  
    int total;  
  
    public int getTotal() {  
        return total;  
    }  
  
    private void g() {  
        // computa valor de total  
        File f = File.open("arq1");  
        f.writeInt(total);  
        ...  
    }  
}
```



```
class A {  
  
    private void f(B b) {  
        int total;  
        total = b.getTotal();  
        ...  
    }  
}
```



Recomendação comum em projeto de software:

Maximize a coesão, minimize o acoplamento

Mas cuidado: minimize (ou elimine) principalmente o acoplamento ruim

Exercícios sobre Propriedades de Projeto

1. Suponha que você ficou responsável pela implementação de um sistema que vai demandar ~100 KLOC.

Proponha hipoteticamente um design para sua implementação que:

- tenha a pior coesão possível
- mas, ao mesmo tempo, tenha o melhor acoplamento possível.

2. (a) Você consegue dizer qual valor o seguinte programa vai imprimir (sem conhecer o código de f)?

```
var g = 1; // global  
f();  
print g;
```



(b) Usando esse programa como exemplo, explique porque variáveis globais dão origem a designs ruins?

(c) Sistemas com muitas variáveis globais não atendem a qual propriedade de projeto?

3. Seja o seguinte código que realiza operações em um conjunto de contas bancárias. (a) Qual a propriedade de projeto é violada por esse código; (b) Como você melhoraria o design desse código?

```
var saldo = [150, 10, 90]; // global

function depositar(conta, valor) {
    saldo[conta] += valor;
}

function getSaldo(conta) {
    return saldo[conta];
}
```



4. Sejam duas classes A e B que:

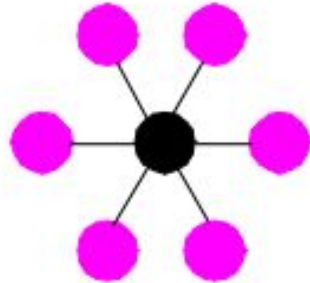
- estão implementadas em diretórios diferentes
- a classe A possui uma referência no seu código para B.

Então, sempre que um programador precisa, como parte de uma tarefa de manutenção, modificar classes A e B que atendem a tais critérios ele conclui a tarefa movendo B para o mesmo diretório de A.

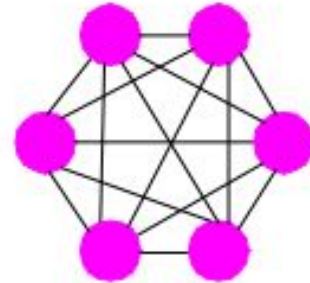
(a) Agindo dessa maneira, o programador estará melhorando qual propriedade de projeto (quando medida entre diretórios)?

(b) E qual propriedade é afetada de modo negativo?

5. Genericamente falando, qual dos seguintes projetos é melhor? Justifique (nodos = classes; arestas = dependências)



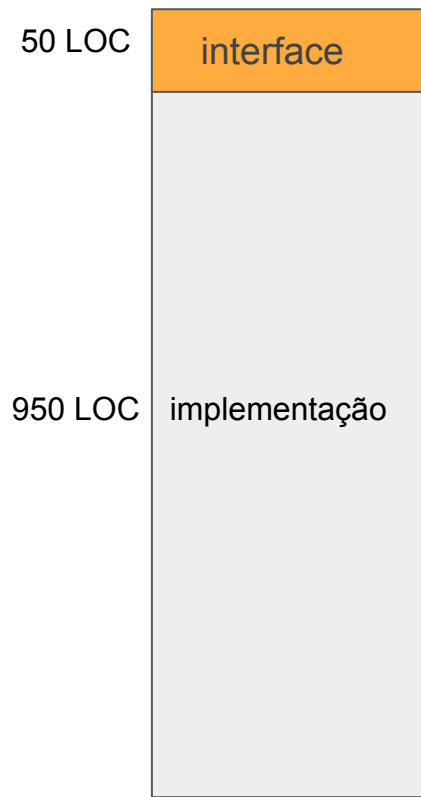
(A)



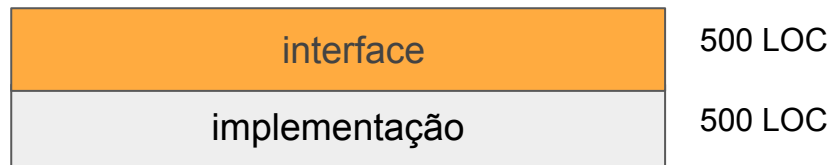
(B)

Fonte: Bertrand Meyer, Object-oriented software construction, 1997 (pág. 47)

6. Qual dos seguintes módulos é melhor? Justifique.



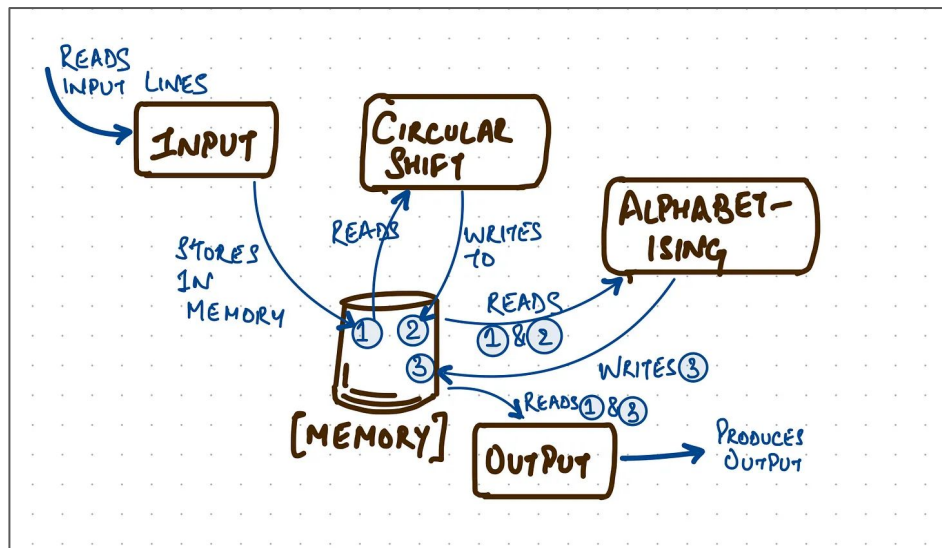
(A)



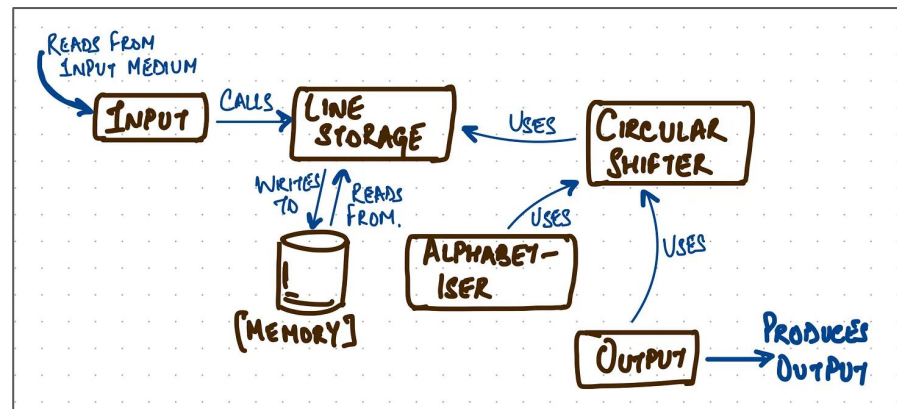
(B)

Inspirado em conceitos propostos no livro A Philosophy of Software Design. John Ousterhout

7. Seguem duas modularizações de um programa que lê um conjunto de linhas da entrada, cria todos os “shift circulares” dessas linhas e depois imprime os shifts em ordem alfabética. Qual das duas modularizações é melhor? Qual propriedade de projeto ela atende (em detrimento da outra)?



Modularização I



Modularização II

Circular Shifts

software engineering is a discipline

engineering is a discipline software

is a discipline software engineering

a discipline software engineering is

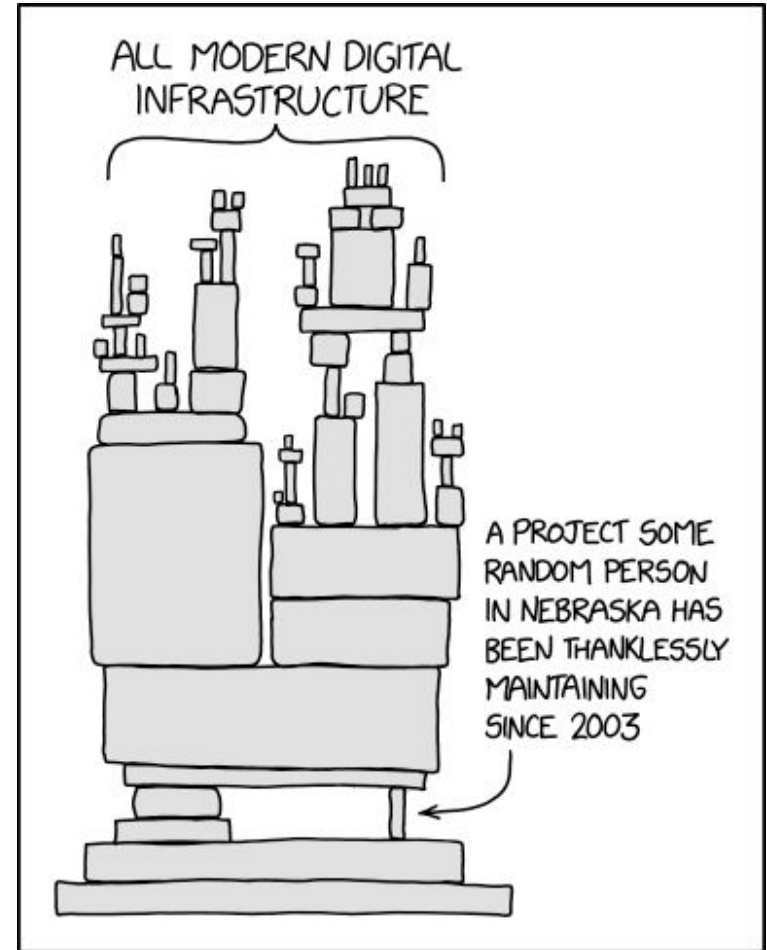
discipline software engineering is a

8. Suponha dois métodos *f* e *g*. Existe um **acoplamento temporal** entre eles quando para chamar *g* temos que chamar antes *f*.

- (a) Dê um exemplo aceitável e bastante comum de acoplamento temporal (ou seja, dê nomes concretos de métodos *f* e *g*).
- (b) O acoplamento temporal do seguinte código é aceitável ou ruim? Se for ruim, sugira uma refatoração para melhorar o código.

```
var circle = new Circle();  
circle.setRadius(5);  
circle.getArea();
```

9. A seguinte figura, de certo modo, ilustra os problemas derivados de qual propriedade de projeto?



Princípios de Projeto

Princípio de Projeto

Responsabilidade Única

Segregação de Interfaces

Inversão de Dependências

Prefira Composição a Herança

Demeter

Aberto/Fechado

Substituição de Liskov

Propriedade de Projeto

Coesão

Coesão

Acoplamento

Acoplamento

Ocultamento de Informação

Extensibilidade

Extensibilidade



Recomendação
mais explícita



Consequência
(o que vamos ganhar
seguindo o princípio)

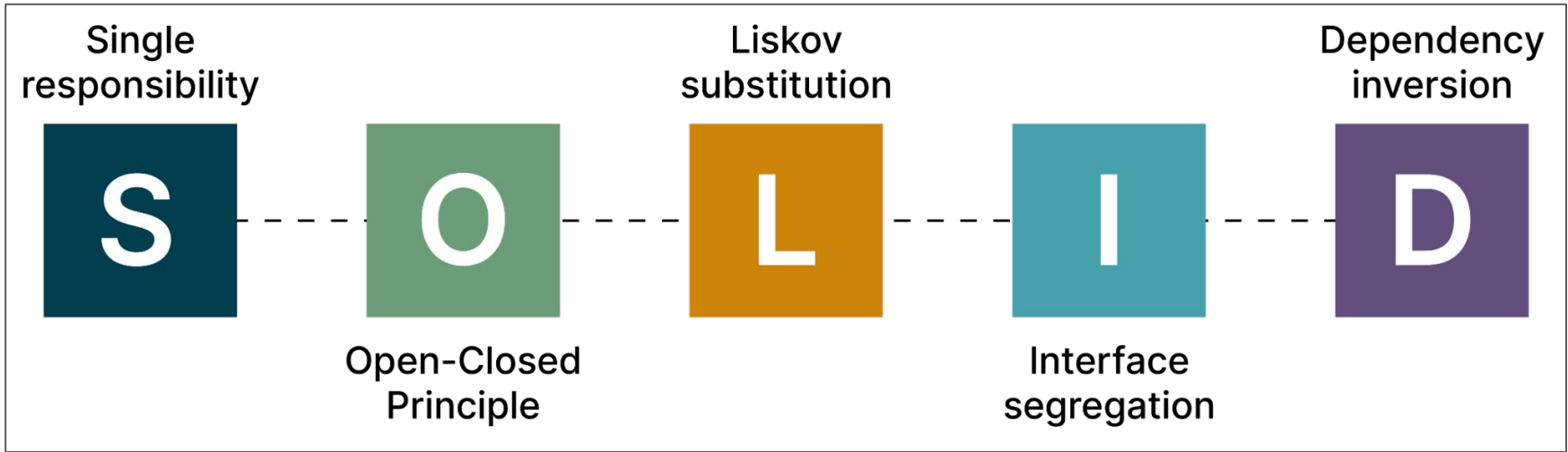


Figura extraída do blog da ThoughtWorks ([link](#))

(1) Princípio da Responsabilidade Única

```
class Disciplina {
```



```
    void calculaIndiceDesistencia() {  
        indice = "calcula índice de desistência"  
        System.out.println(indice);  
    }
```

```
}
```

Responsabilidade #1: **calcular** índice de desistência

```
class Disciplina {  
  
    void calculaIndiceDesistencia() {  
        indice = "calcula índice de desistência"  
        System.out.println(indice);  
    }  
  
}
```



Responsabilidade #2: **imprimir** índice de desistência

Versão com **separação de responsabilidades**

```
class Console {
```



```
    void imprimeIndiceDesistencia(Disciplina disciplina) {  
        double indice = disciplina.calculaIndiceDesistencia();  
        System.out.println(indice);  
    }
```

```
}
```

```
class Disciplina {
```

```
    double calculaIndiceDesistencia() {  
        double indice = "calcula índice de desistência"  
        return indice;  
    }
```

```
}
```

Classe de apresentação (interface com usuário)
⇒ desenvolvedor frontend

```
class Console {  
  
    void imprimeIndiceDesistencia(Disciplina disciplina) {  
        double indice = disciplina.calculaIndiceDesistencia();  
        System.out.println(indice);  
    }  
  
}  
  
class Disciplina {  
  
    double calculaIndiceDesistencia() {  
        double indice = "calcula índice de desistência"  
        return indice;  
    }  
  
}
```



Classe de negócio (regra de negócio) ou classe de domínio
⇒ desenvolvedor backend
⇒ mais fácil de ser testada (teste de unidade)

Violando SRP em 1 slide



(2) Princípio da Segregação de Interfaces

Segregação de Interfaces

- Interfaces devem ser pequenas, coesas e específicas para cada tipo de cliente
- Caso particular do princípio anterior, mas voltado para interfaces

```
interface Funcionario {  
    double getSalario();  
  
    double getFGTS(); // apenas funcionários CLT  
  
    int getSIAPE(); // apenas funcionários públicos  
  
    ...  
}
```



Versão que atende **segregação de interfaces**



```
interface Funcionario {  
    double getSalario();  
    ...  
}
```

```
interface FuncionarioCLT extends Funcionario {  
    double getFGTS();  
    ...  
}
```

```
interface FuncionarioPublico extends Funcionario {  
    int getSIAPE();  
    ...  
}
```

(3) Princípio da Inversão de Dependências

Inversão de Dependências

- Na verdade, vamos chamar esse princípio de "**Prefira Interfaces a Classes**"
- Pois transmite melhor a sua ideia!

Exemplo sem usar Inversão de Dependências

```
class ControleRemoto {  
    TVsamsung tv;  
    ...  
}  
  
class TVsamsung {  
    ...  
}
```



Qual o possível
problema de design?

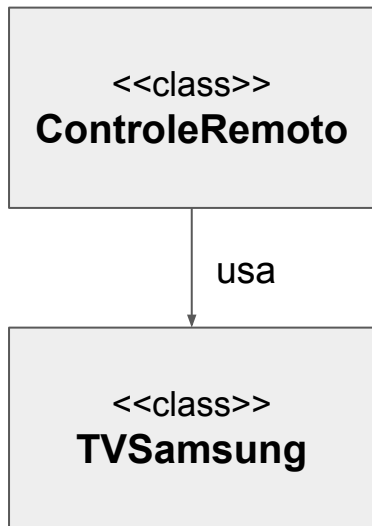
Exemplo usando Inversão de Dependências

```
class ControleRemoto {  
    TVGenerica tv;  
    ...  
}
```

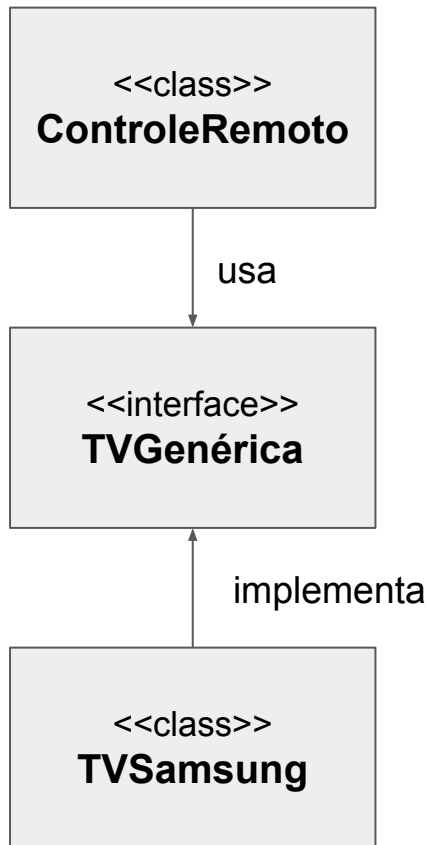
```
interface TVGenerica {  
    ...  
}
```

```
class TVSamsung implements TVGenerica {  
    ...  
}
```

Sem DIP



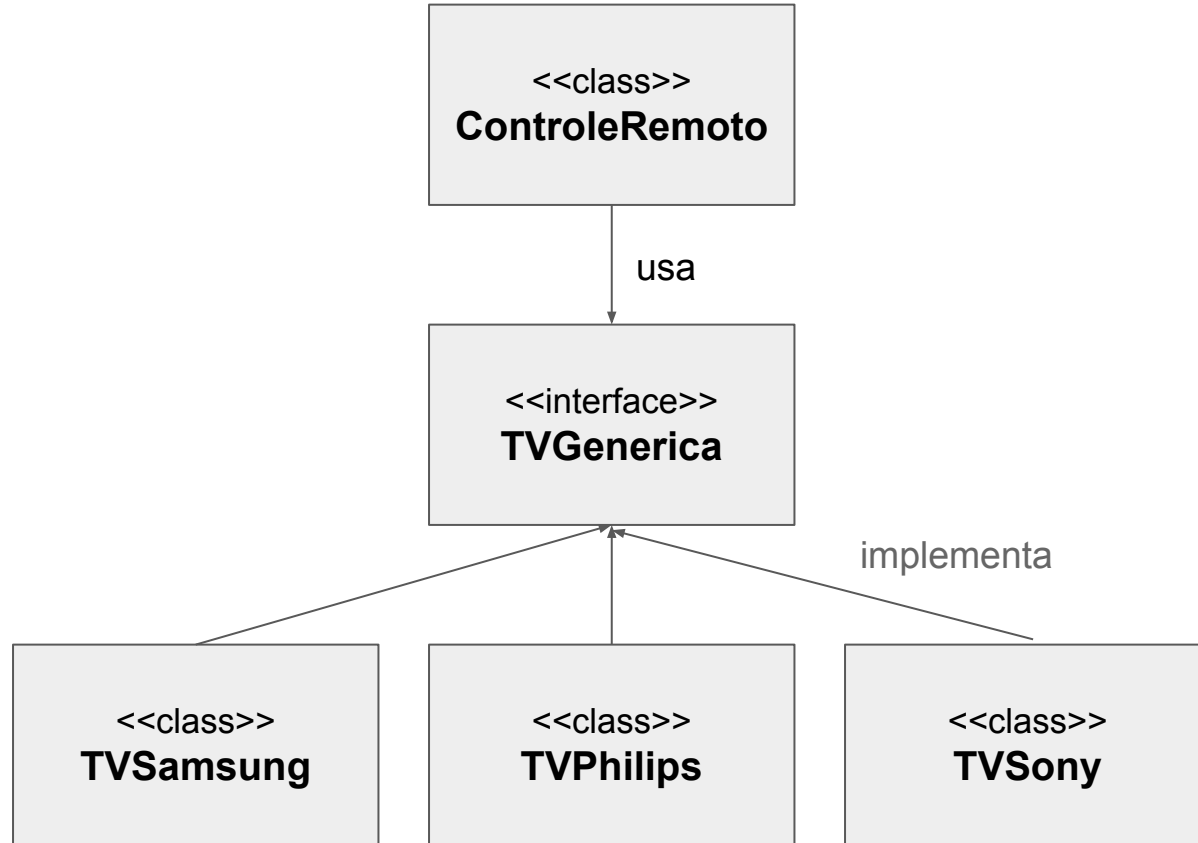
Com DIP



Vantagem: controle remoto genérico, para qualquer TV

Posso mudar de TV, sem alterar o código da classe ControleRemoto

Design após algum tempo



(4) Prefira Composição a Herança

Contexto Histórico

- Na década de 80, quando orientação a objetos tornou-se popular, as pessoas começaram a abusar de herança
- Achavam que herança iria ser uma bala de prata, promover reúso em larga escala, etc.

Herança: relação do tipo "é-um"

```
class MotorGasolina extends Motor {  
    ... // herda atributos e métodos de motor  
}
```



Composição: relação do tipo "possui"

```
class Painel {  
    ContaGiros cg;  
    . . .  
}
```



Prefira Composição a Herança $\Rightarrow$ não force o
uso de herança

(5) Princípio de Demeter

Demeter

- Demeter: nome de um grupo de pesquisa americano
- Evite longas cadeias de chamadas de métodos
- Exemplo:

```
obj.getA().getB().getC().getD().getOqueEuPreciso();
```



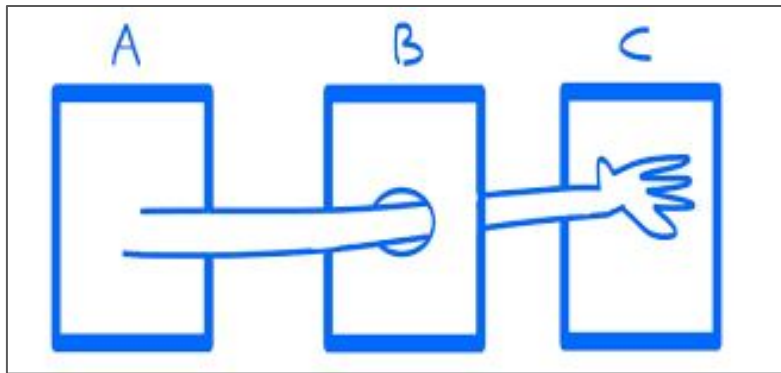
objetos de passagem



Motivo

- Longas cadeias de chamadas quebram encapsulamento
- Não quero passar por A, B, C,... até obter que eu preciso

Violando Demeter



<https://medium.com/@evan.hopkins.us/the-law-of-demeter-and-its-application-to-react-ab1e054f13c5>

```
class PrincipioDemeter {  
  
    T1 attr;  
  
    void f1() {  
        ...  
    }  
  
    void m1(T2 p) { // método que segue Demeter  
        f1();           // caso 1: própria classe  
        p.f2();          // caso 2: parâmetro  
        new T3().f3();    // caso 3: criado pelo método  
        attr.f4();        // caso 4: atributo da classe  
    }  
  
    void m2(T4 p) { // método que viola Demeter  
        p.getX().getY().getZ().doSomething();  
    }  
}
```



Mas cuidado



- Demeter e demais princípios são recomendações
- Não devemos ser radicais e achar que cadeias de chamadas de métodos são totalmente proibidas
- Casos isolados podem existir e terem uma boa justificativa

Exemplo justificável de cadeias de métodos

```
const numbers = [1, 2, 3, 4, 5, 6];  
const result = numbers  
  .map(num => num * 2)  
  .filter(num => num % 3 === 0)  
  .reduce((acc, num) => acc + num, 0);  
console.log(result); // Output: 18
```



Métodos da linguagem
ou de sua API

(6) Princípio Aberto/Fechado

Princípio Aberto/Fechado

- Proposto por Bertrand Meyer
- Ideia: uma classe deve estar **fechada** para modificações, mas **aberta** para extensões



Explicando melhor

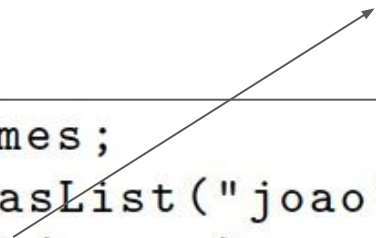
- Suponha que você vai implementar uma classe
- Usuários ou clientes vão querer usar a classe (óbvio!)
- Mas vão querer também customizar, parametrizar, configurar, flexibilizar e estender a classe!
- Você deve se antecipar e tornar possível tais extensões
- Mas sem que os clientes tenham que editar o código da classe

Como tornar uma classe **aberta** a extensões, mas mantendo o seu código **fechado** para modificações?

- Parâmetros
- Funções de mais alta ordem
- Padrões de projeto
- Herança
- etc

Exemplo

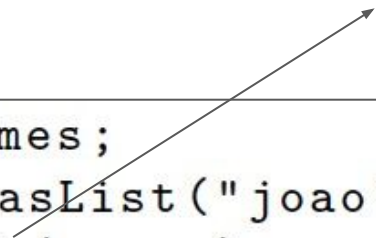
Ordena a lista passada com parâmetro



```
List<String> nomes;  
nomes = Arrays.asList("joao", "maria", "alexandre", "ze");  
Collections.sort(nomes);
```

Exemplo

Ordena uma lista passada com parâmetro



```
List<String> nomes;  
nomes = Arrays.asList("joao", "maria", "alexandre", "ze");  
Collections.sort(nomes);
```

```
System.out.println(nomes);  
// resultado: ["alexandre","joao","maria","ze"]
```


Mas agora eu quero ordenar as strings da lista pelo seu tamanho, isto é, pelo número de chars

Será que o método sort está aberto (preparado) para permitir essa extensão?

Mas, mantendo o seu código fechado, isto é, sem ter que mexer no seu código

Solução

lista que será
ordenada

Função usada por sort para comparar s1 e s2:

- $\text{result} < 0 \Rightarrow$ ordem é s1, s2
- $\text{result} = 0 \Rightarrow$ tanto faz a ordem
- $\text{result} > 0 \Rightarrow$ ordem é s2, s1

```
Collections.sort(nomes, (s1, s2) -> s1.length() - s2.length());
```

```
System.out.println(nomes);
```

```
// result: ["ze", "joao", "maria", "alexandre"]
```



Não existe almoço grátis: cliente (código que chama "sort") teve que implementar e passar como parâmetro uma função que define o critério a ser usado na ordenação

Resumindo: ao implementar uma classe, pense em pontos de extensão!

(7) Princípio de Substituição de Liskov

Princípio de Substituição de Liskov

- Nome é uma referência à Profa. Barbara Liskov
- Princípio define boas práticas para uso de herança
- Especificamente, boas práticas para redefinição de métodos em subclasses



Primeiro: vamos entender o termo "substituição"

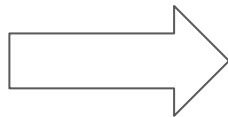
```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```



```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```

```
f(new B1()); // f pode receber objetos da subclasse B1  
...  
f(new B2()); // e de qualquer outra subclasse de A, como B2  
...  
f(new B3()); // e B3
```

```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```



- Tipo A pode ser substituído por B1, B2, B3,...
- Desde que eles sejam subclasses de A

Princípio de Substituição de Liskov

- Substituições de A por B podem acontecer desde que B ofereça os mesmos serviços de A
- Ou seja: para um código feito para usar A, a substituição de A por B é “imperceptível”

Exemplo que segue Substituição de Liskov

```
class ControleRemoto {  
    // alcance de 10 m  
}
```

```
class ControleRemotoPremium extends ControleRemoto {  
    // alcance de 20 m  
}
```



Exemplo que **não** segue Substituição de Liskov

```
class ControleRemoto {  
    // alcance de 10 m  
}
```

```
class ControleRemotoXYZ extends ControleRemoto {  
    // alcance de 5 m  
}
```



Exercícios sobre Princípios de Projeto

1. Qual princípio de projeto é violado por uma chamada como a mostrada abaixo? Qual mudança de projeto você faria na classe `Biblioteca` (tipo de `bib`) para remover essa violação?

```
bib.getAcervo().getAreaConhecimento("ES").  
    getLivros().find("ESM").getNumExemplares();
```

2. Suponha a seguinte classe:

```
class Tabela {  
    ...  
    void imprime() {  
        // código que imprime cabeçalho  
        // código que imprime cada linha da tabela  
        // código que imprime rodapé  
    }  
    ...  
}
```

Esse código não segue o Princípio Aberto/Fechado, pois no nosso sistema é importante poder **variar as mensagens de cabeçalho e rodapé**. Como você mudaria a implementação dessa classe para atender ao Princípio Aberto/Fechado?

3. Suponha uma classe Calculadora com um método que verifica se um número entre 0 e 10000 é primo. Suponha ainda que um algoritmo mais eficiente para essa tarefa foi implementado em uma subclasse CalculadoraRapida. No entanto, ele somente é capaz de trabalhar com números entre 1000 e 9000.

```
class Calculadora {  
    boolean isPrimo(n) {  
        // 0 <= n < 10000  
    }  
}
```

```
class CalculadoraRapida extends Calculadora {  
    boolean isPrimo(n) {  
        // 1000 <= n < 9000  
    }  
}
```

Qual princípio de projeto é violado nessa implementação? Justifique.

4. Suponha que você concluiu um curso de extensão oferecido pela UFMG e quer receber seu certificado. Para isso, você teve que:

- Mandar uma msg para o coordenador do curso, que te pediu para mandar uma msg para secretaria do DCC.
- Então, você mandou uma msg para a secretaria do DCC, que te pediu para mandar uma mensagem para o CENEX.
- Então, você mandou uma msg para o CENEX, que te pediu para mandar uma msg para a PROEX.
- Então, você mandou uma msg para a PROEX, que finalmente retornou o seu certificado.

Essa sequência de passos ilustra uma violação de qual princípio de projeto? Além de ter que mandar várias mensagens, qual um outro

¹²²problema dessa “solução”?

5. Suponha que você está implementando um sistema S e precisa usar um serviço de processamento de pagamentos X para, por exemplo, aceitar pagamentos via cartões de crédito.

No entanto, você não quer acoplar S a X, pois futuramente você pode optar por um outro serviço Y.

- (a) Explique como você faria para desacoplar S de X.
- (b) Qual princípio de projeto você usou para garantir esse desacoplamento?

6. Em Engenharia de Software, às vezes usamos soluções mais complexas em contextos nos quais isso não é necessário. Esse problema é chamado de **overengineering** ou **otimização prematura**.

Então crie e descreva um contexto no qual o uso de um dos princípios de projeto que estudamos é uma otimização prematura.

Se quiser, você pode criar e explicar esse contexto usando o mesmo exemplo que usamos na explicação do princípio.

Fim